

Лекция 1. Введение и воспоминания о ФП

ФП 2.0 · осень 2026

Андрей Стоян

осень 2026

План на сегодня

- **О курсе:** зачем он, как устроен и что в нём будет, глава за главой
- **Вспоминаем ФП:** глава 1 конспекта по разделам, с задачами
 - термы и редукция · типы · функции, данные, классы типов и монады в Haskell

О курсе

Фольклор, который никто не рассказывает

- Многие сложные вещи в дизайне языков и программ — частные случаи нескольких простых принципов
- Эти принципы считаются «общеизвестным фольклором», поэтому их не объясняют
- — Знание рассеяно по книгам, статьям и культовым блог-постам; собрать картину самому — долго
- ⇒ **Цель курса** — собрать фольклор в одном месте и организовать в систему
- ⇒ Курс явно опирается на классические работы; **чтение статей — часть работы в курсе**

Что здесь значит «функциональное»

- **Δ** **ФП** — трепетное отношение к понятию *эффекта*
 - в других школах мысли эффект — аксиоматическая данность
 - в ФП — предмет изучения, сознательного конструирования и аккуратного обращения
- **⇒** Это полезно для изучения языков и построения выразительных конструкций
- **⇒** И это основа продуктивного стиля программирования
- **i** Функциональные языки сравнительно просты, поэтому новые идеи часто рождаются в них и расходятся дальше
- **Haskell** как язык курса: воплощает многие концепции, доведённые до логического завершения, достаточно выразителен, чтобы кодировать остальные, и при этом промышленный

Широта против глубины

Курс широкий, но не всегда глубокий

- детали GHC и теоретико-категорные основания — в общем виде
- языковой дизайн через оптику ФП — максимально подробно
- **NB Пререквизиты:** АТД, паттерн-матчинг, свёртки, параметрический полиморфизм, классы типов, базовые монады
- **+** Полезно: читать типовые дробы, полиморфное λ -исчисление, кодирование Чёрча

Курс для вас, если вы

- интересуетесь дизайном языков и красивыми обобщениями
- хотите лучше проектировать API
- планируете работать на функциональных языках
- просто хотите знать, как вообще бывает

Как устроен курс

- **Занятие раз в неделю:** два часа с перерывом, лекция и практика вперемешку
- **Конспект — источник истины;** слайды и записи — производные. Главу читать до или сразу после занятия
- **Домашка каждую неделю**
 - маленький обязательный минимум — допуск к экзамену (все минимумы сданы, не больше двух пропусков)
 - остальные задачи — уровень «к экзамену» и «челлендж», по желанию
 - проверка автоматическая: тесты и линтер в CI, ревью от LLM — совещательное
- **Оценка за курс = экзамен:** вопрос по конспекту и must-read главы плюс задача из домашних уровня «к экзамену»
- **Банк вопросов по главам** публикуется с первой недели — это и программа экзамена, и вопросы для повторения

Что будет в курсе

Глава 2. Параметрический полиморфизм

- Какие формы принимает полиморфизм в языках и как он реализуется: мономорфизация, словари, боксинг
- System F, ранги, кодирование данных полиморфными функциями
- **Тизер:** структуры данных на уровне типов и «полиморфизм по полиморфизму» — по конвенции вызова

```
data HList (ts :: [Type]) where
  HNil    :: HList '[]
  (:&)    :: t -> HList ts -> HList (t ': ts)

($) :: forall r a (b :: TYPE r). (a -> b) -> a -> b
```

- ➔ Полиморфизм — код, который ничего не знает о типе; на этом стоит модульность интерпретаторов

Глава 3. Ad-hoc полиморфизм

- Как устроены классы типов: словари, резолв инстансов как логическое программирование
- Семейства типов — функции на типах
- **Тизер:** 100500 применений классов типов, и многое окажется их частным случаем: рефлексия, `Typeable`, `Generic`, зависимость от значения через тип

```
class Collection c where  
  type Elem c  
  insert :: Elem c -> c -> c
```

- ➔ Классы типов — механизм расширяемости; выстрелит в `tagless final` и в системах эффектов

Глава 4. Типы данных

- Алгебра типов и изоморфизмы: ищем дизайн данных алгебраически
- Вариантность и при чём тут функторы
- Рекурсивные типы через неподвижную точку; свёртки и развёртки как универсальные схемы
- **Тизер:** всё через свёртки? всё через развёртки? как ФП связано с ООП?

```
newtype Fix f = In (f (Fix f))  
cata :: Functor f => (f a -> a) -> Fix f -> a
```

- ➔ Здесь впервые появляется **expression problem** — главная проблема курса

Глава 5. Интерпретаторы

- Научимся видеть интерпретаторы повсюду: башня интерпретаторов, встроенные языки
- Типы во встроенных языках, связывания и функции: интерпретируем самих себя самими собой
- Tagless final — интерпретатор без синтаксического дерева
- **Тизер:** почему написание функции — это расширение языка? монада — это интерпретатор чего, простите?

```
class ExprSym repr where
  lit :: Int -> repr Int
  add :: repr Int -> repr Int -> repr Int
```

- ➡ Expression problem формулируется точно: добавить и конструкцию, и интерпретацию, не трогая старое

Глава 6. Продолжения

- Как понять продолжения: «остаток вычисления» как значение
- CPS и стейт-машины; `Cont` — one monad to rule them all
- Дефункционализация продолжений — и мы получаем абстрактную машину
- Ограниченные продолжения, генераторы, корутины
- **Тизер:** `callCC`, `shift/reset`, и почему исключения, `async` и `yield` — одно и то же

```
newtype Cont r a = Cont { runCont :: (a -> r) -> r }
```

- ➡ Продолжения — материал, из которого сделаны хендлеры эффектов

Глава 7. Эффекты, хендлеры, трансформеры

- Алгебраические эффекты: операция отдельно, интерпретация отдельно
- Free-монады, data types à la carte, трансформеры монад — три способа собрать интерпретатор из кусков
- **Тизер:** любой язык программирования выражается через одну конструкцию?

```
data Free f a = Pure a | Free (f (Free f a))
```

- ➡ Это ответ на expression problem: модульные интерпретаторы

Глава 8. Системы эффектов

- Как типизировать эффекты: строки эффектов, хендлеры в типах
- Koika и Effekt как языки; `effectful`, `polysemy`, `bluefin` как библиотеки
- **Тизер:** типизированные ошибки без боли — это возможно? Наверное

```
fun divide(a : int, b : int) : exn int
```

- ➡ То, чем сегодня живёт промышленный Haskell, и куда движутся другие языки

Главы 9–10. Обзорные

Datatype-generic programming

- Как не писать код, когда его можно не писать
- `deriving` изнутри: `stock`, `newtype`, `via`, `anyclass`
- `GHC.Generics` и специализация
- **Тизер:** `deriving via` уже в первой домашке — сможете объяснить, что там происходит

Персистентность и оптика

- Персистентные структуры данных по цене мутлирующих
- `Zipper` — курсор в неизменяемой структуре
- Линзы, призмы, траверсалы — «пути» как первоклассные значения
- **Тизер:** `over (each . _Just)` (+1)

Вспоминаем ФП

Термы и редукция

Программы — это выражения

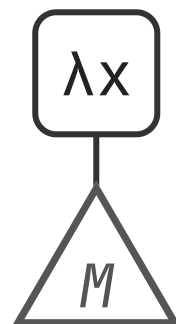
- В ФП программа — выражение; выполнение — **редукция** выражения к более простому
- Выражение можно записать строкой, а можно деревом: дереву не нужны скобки и приоритеты
- **Δ** Простейший функциональный язык — λ -исчисление; его выражения — λ -термы из вершин трёх видов

$$\Lambda ::= V \mid (\lambda V. \Lambda) \mid (\Lambda \Lambda)$$

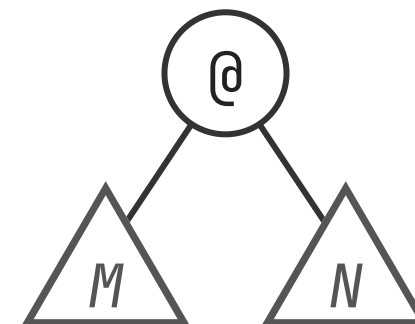
Переменная $x \in V$



Абстракция $(\lambda x. M)$



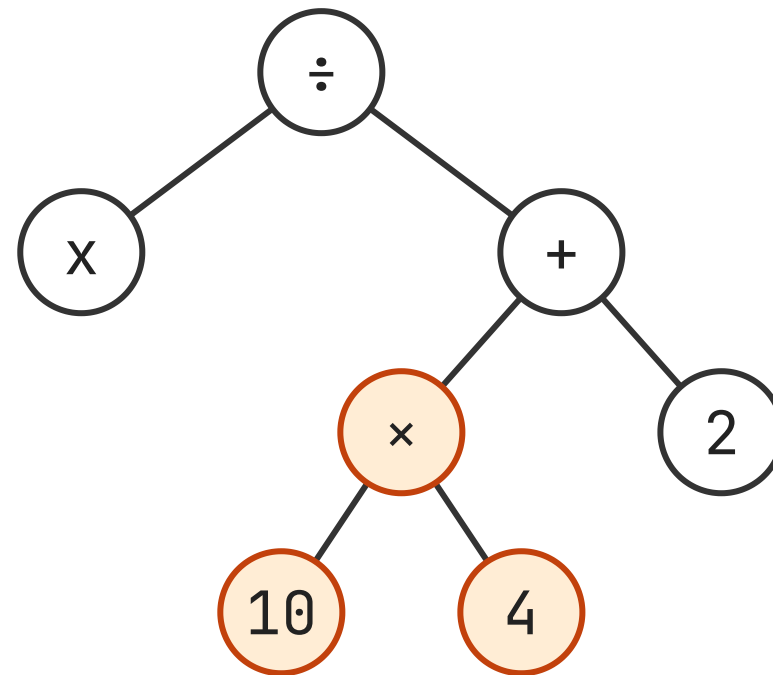
Аппликация $(M N)$



Из выражения в функцию и обратно

Есть выражение. Хотим переиспользовать его для разных значений вместо 10×4 .

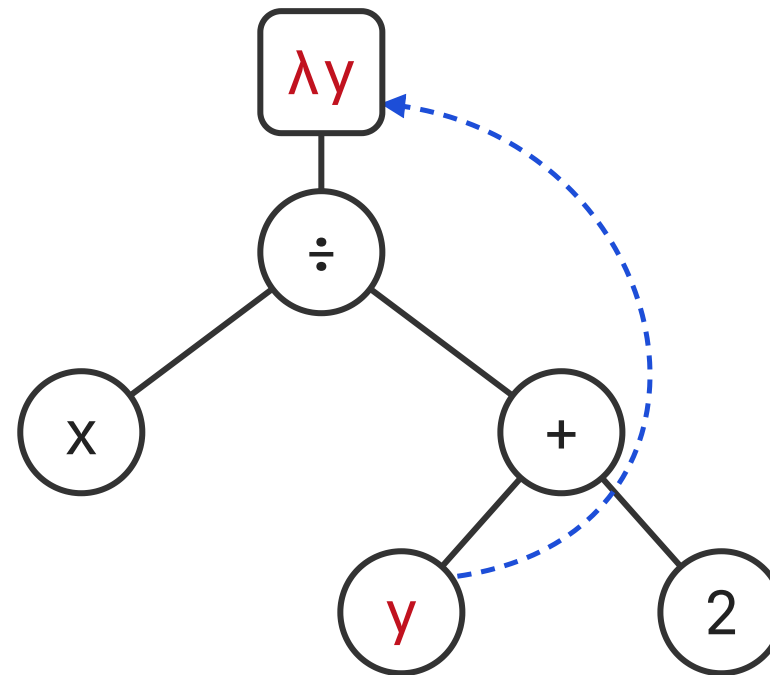
$$x \div (10 \times 4 + 2)$$



Из выражения в функцию и обратно

Заменяем фрагмент **формальным параметром** y и объявляем его вершиной λ выше по дереву.
Получилась функция одного аргумента.

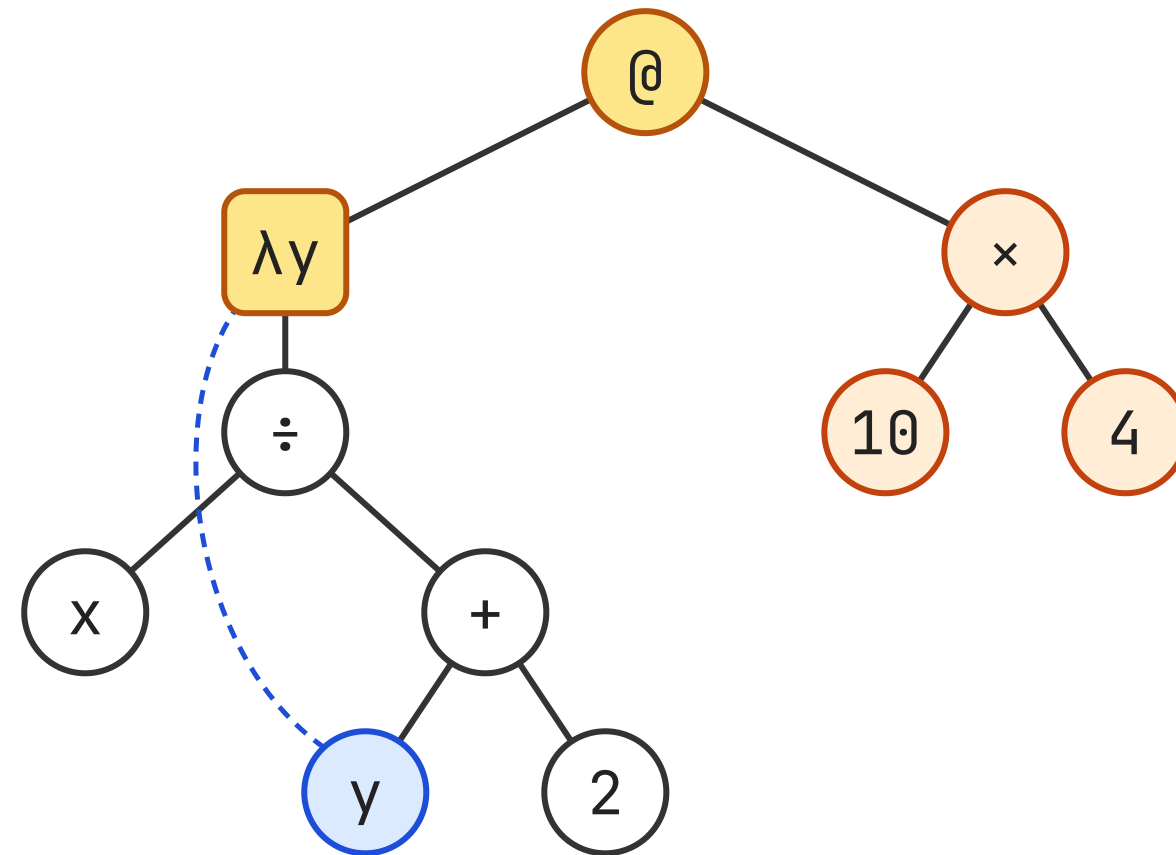
$$\lambda y. x \div (y + 2)$$



Из выражения в функцию и обратно

Вершина-аппликация @ подставляет **конкретный аргумент**. Аппликация абстракции к аргументу — **редекс**.

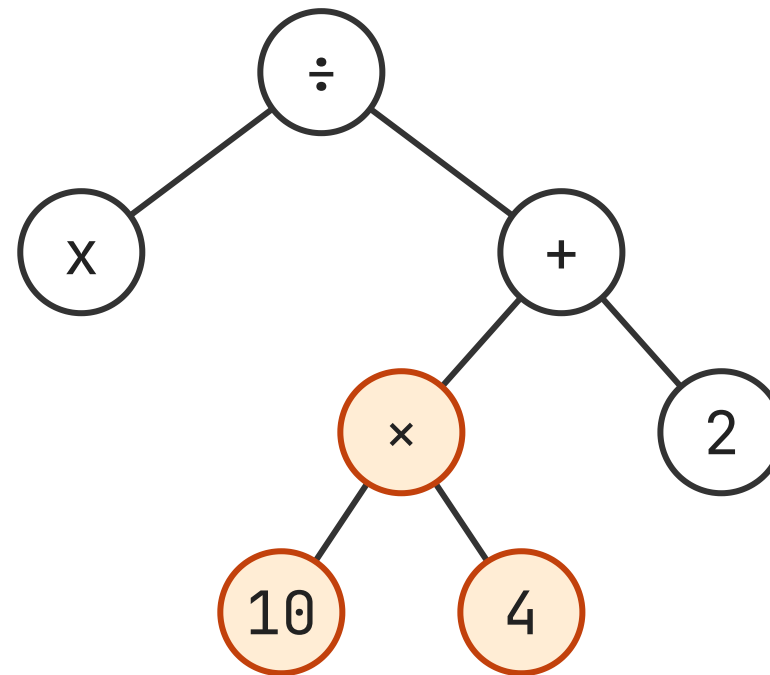
$(\lambda y. x \div (y + 2)) (10 \times 4)$



Из выражения в функцию и обратно

Редукция: аргумент подставлен во все свободные вхождения y в теле, вершины $@$ и λ исчезли. Мы вернулись к исходному выражению.

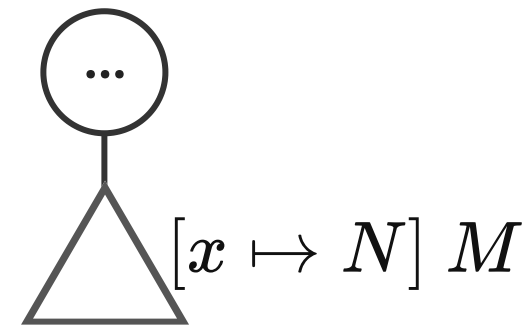
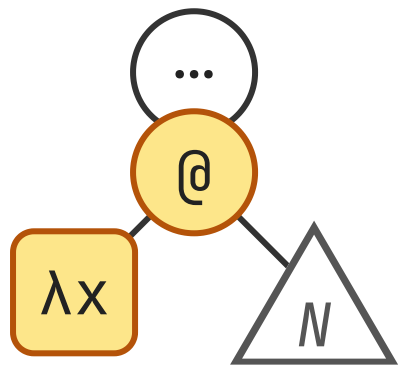
$$x \div (10 \times 4 + 2)$$



Правило редукции

- **Δ β-редукция** — единственное правило переписывания: найти применение λ-функции к аргументу и подставить аргумент вместо связанной переменной

$$(\lambda x. M) N \rightsquigarrow_{\beta} [x \mapsto N] M$$



Задача: редукция

✎ Найдите редексы и доведите терм до нормальной формы, считая $+$ и числа примитивами:
 $(\lambda f x. f (f x)) (\lambda y. y + 1) 0$. Сколько понадобилось β -шагов?

⚡ **Решение.**

$$\begin{aligned} & (\lambda f x. f (f x)) (\lambda y. y + 1) 0 \\ \rightsquigarrow_{\beta} & (\lambda x. (\lambda y. y + 1) ((\lambda y. y + 1) x)) 0 \\ \rightsquigarrow_{\beta} & (\lambda y. y + 1) ((\lambda y. y + 1) 0) \\ \rightsquigarrow_{\beta} & ((\lambda y. y + 1) 0) + 1 \\ \rightsquigarrow_{\beta} & (0 + 1) + 1 \end{aligned}$$

Три β -шага, дальше только арифметика.

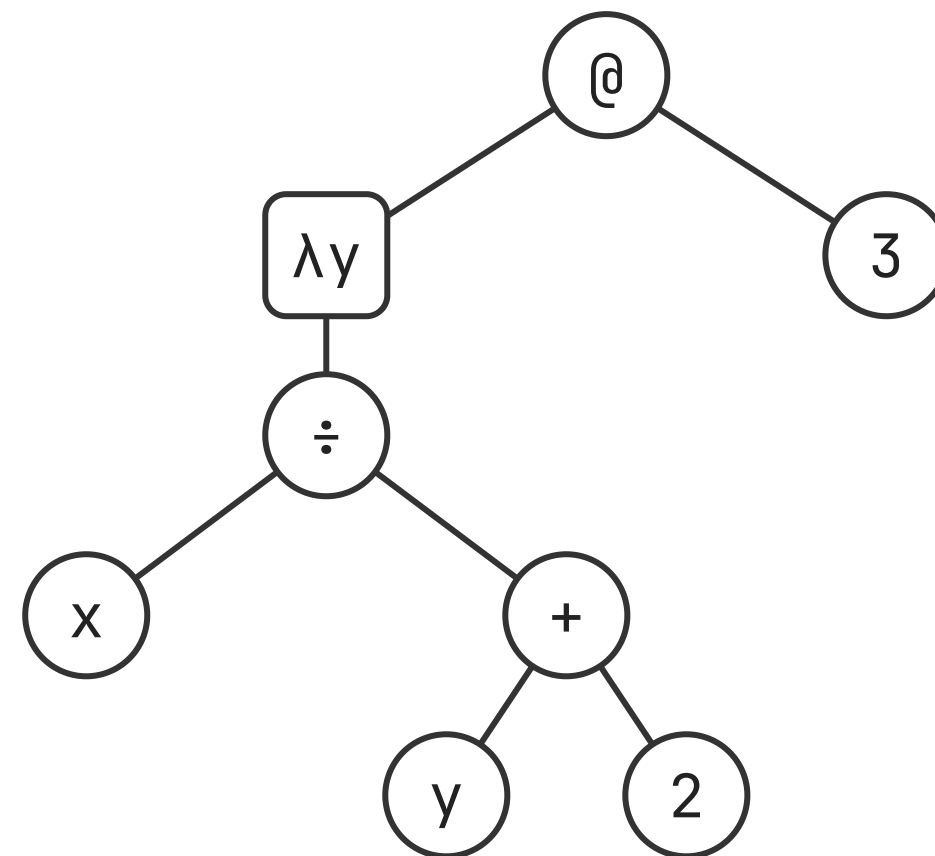
Типы

Зачем типы

- Программное обеспечение — это сложно, поэтому ошибки в программах неизбежны
- Их можно искать **статически**, не запуская программу; анализ типов — один из видов статического анализа
- **Δ Идея:** каждой вершине дерева программы приписываем синтаксическую **метку** по правилам
- Если удалось приписать метку каждой вершине — программа проходит проверку типов и считается «хорошей»

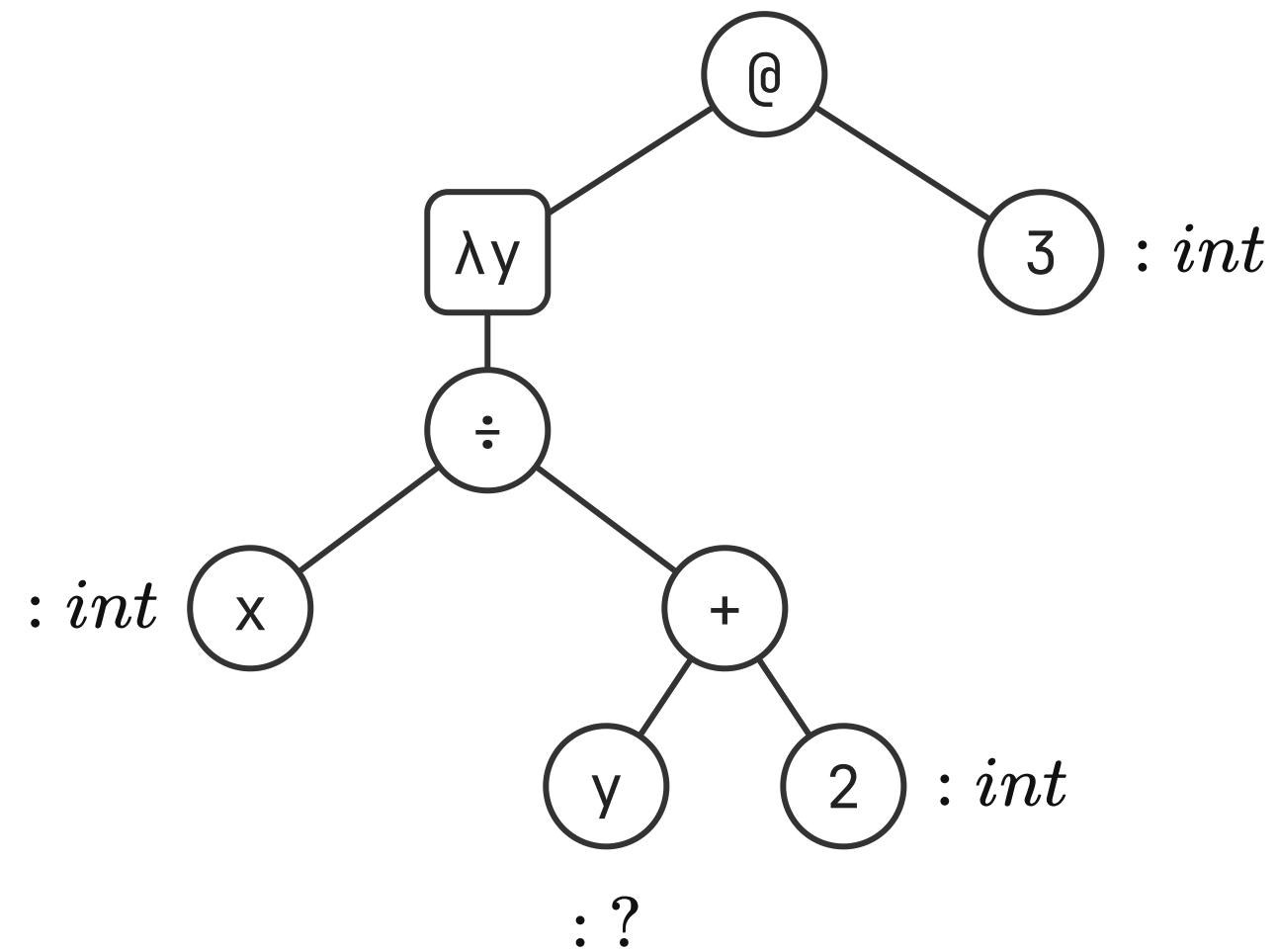
Приписываем типовые метки

Терм $(\lambda y. x \div (y + 2))\ 3$ в контексте, где x — целое. Приписываем метки снизу вверх.



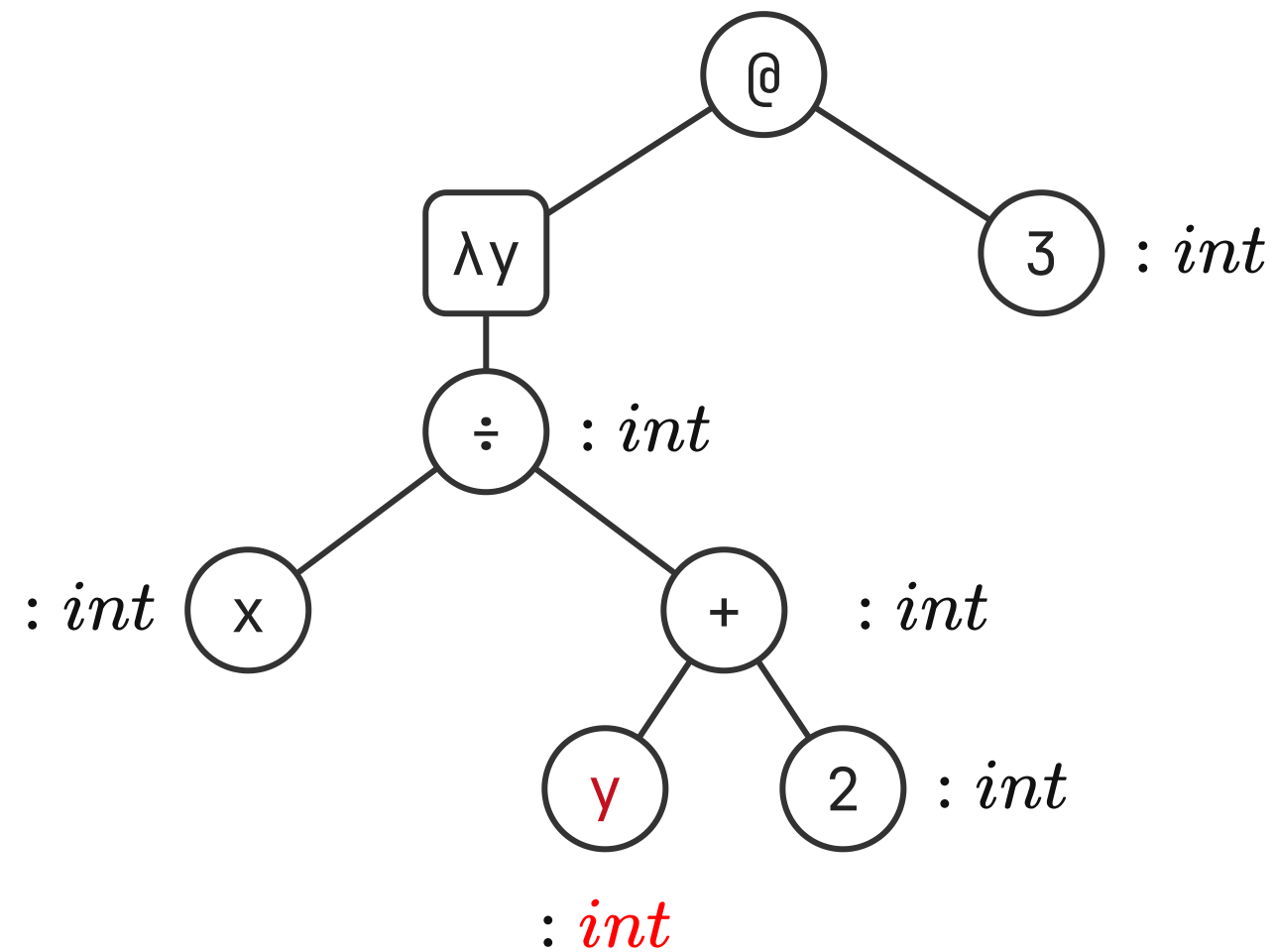
Приписываем типовые метки

Листья: константы и переменные из контекста. Про y пока ничего не известно.



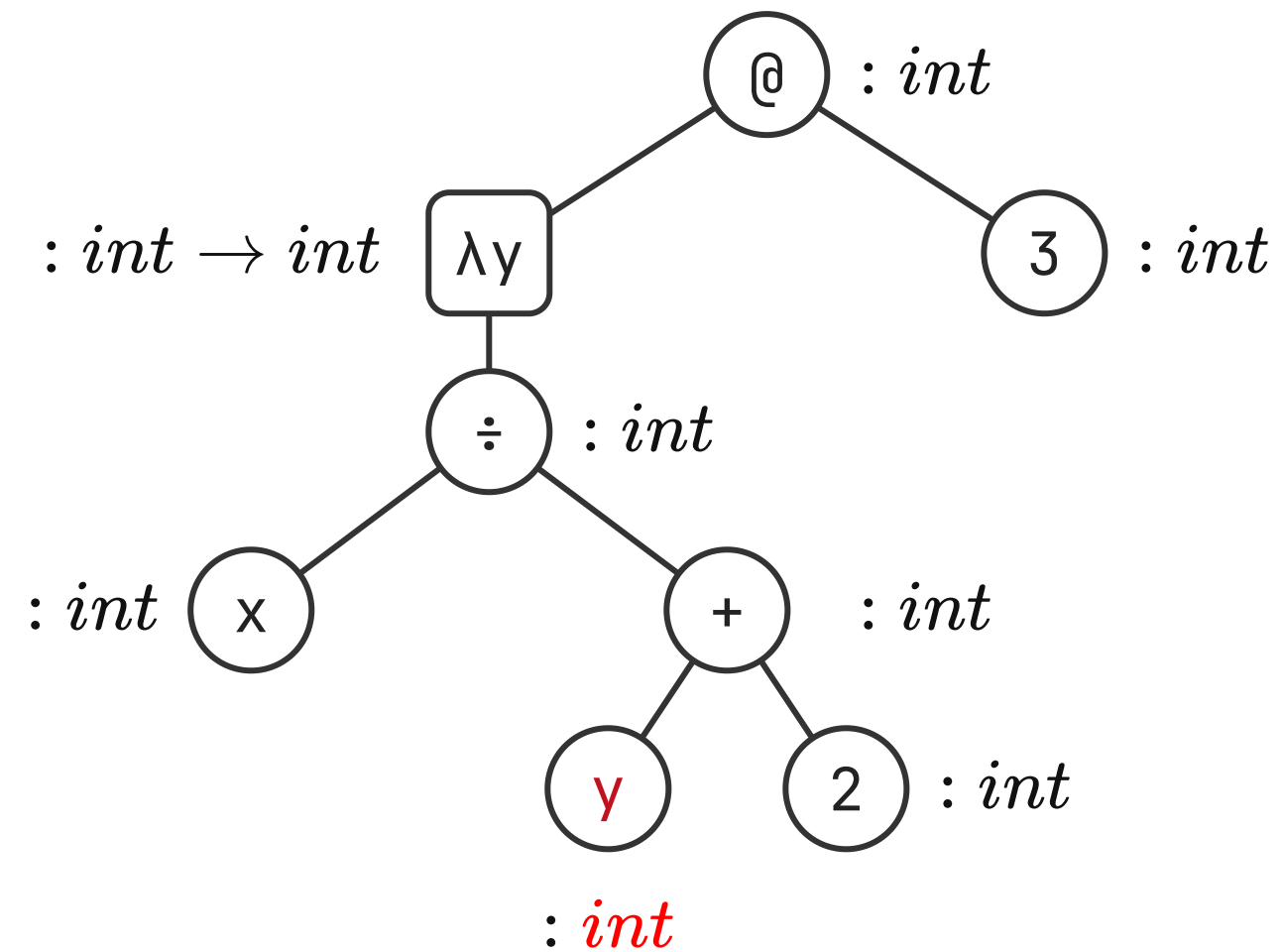
Приписываем типовые метки

Арифметика требует целых с обеих сторон: значит, *y* **обязана** быть целым — метку *y* параметра мы угадали, а не вывели.



Приписываем типовые метки

Абстракция получает стрелочный тип, аппликация — его результат. Все вершины помечены: терм типизируется.



Система типов

- **Δ** Система типов задаёт синтаксис меток и правила, по которым их можно приписывать
- Синтаксис — в нотации вроде BNF; правила — **типовые дроби**: над чертой посылки, под чертой вывод
- Просто типизированное λ -исчисление $\lambda_{\rightarrow}$:

$$\frac{(x : \sigma) \in \Gamma}{\Gamma \vdash x : \sigma} \text{ctx} \quad \frac{\{x : \sigma\} \cup \Gamma \vdash M : \tau}{\Gamma \vdash \lambda x^{\sigma}. M : \sigma \rightarrow \tau} \text{intro}_{\rightarrow} \quad \frac{\Gamma \vdash M : \sigma \rightarrow \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash M N : \tau} \text{elim}_{\rightarrow}$$

- **i** Метки чисто синтаксические, но их можно **интерпретировать**: самая популярная интерпретация — тип как множество, $int \rightarrow int$ как множество функций $\mathbb{Z} \rightarrow \mathbb{Z}$

Дерево вывода по кликам

Проверим, что $\lambda f x. f x$ имеет тип $(\sigma \rightarrow \tau) \rightarrow \sigma \rightarrow \tau$. Здесь $\Gamma = \{f : \sigma \rightarrow \tau, x : \sigma\}$.

$$\frac{\frac{\frac{}{\Gamma \vdash f : \sigma \rightarrow \tau} \text{ctx} \quad \frac{}{\Gamma \vdash x : \sigma} \text{ctx}}{\Gamma \vdash f x : \tau} \text{elim} \rightarrow}{\{f : \sigma \rightarrow \tau\} \vdash \lambda x. f x : \sigma \rightarrow \tau} \text{intro} \rightarrow}{\emptyset \vdash \lambda f x. f x : (\sigma \rightarrow \tau) \rightarrow \sigma \rightarrow \tau} \text{intro} \rightarrow$$

ШАГИ

1. intro $\rightarrow$: снимаем λf
2. intro $\rightarrow$: снимаем λx
3. elim $\rightarrow$: $f x$ — две ветки
4. ctx: f из Γ
5. ctx: x из Γ — листья закрыты

Дерево вывода — перевёрнутое дерево терма: каждой вершине по одному правилу.

Задача: тип и дерево вывода

✎ Постройте дерево вывода и определите тип терма $\lambda x y. y x$. Какая функция из Prelude имеет такой тип?

⚡ **Решение.**

$$\frac{\frac{\frac{\overline{\Gamma \vdash y : \sigma \rightarrow \tau}^{ctx} \quad \overline{\Gamma \vdash x : \sigma}^{ctx}}{\Gamma \vdash y x : \tau}^{elim \rightarrow}}{\{x : \sigma\} \vdash \lambda y. y x : (\sigma \rightarrow \tau) \rightarrow \tau}^{intro \rightarrow}}{\emptyset \vdash \lambda x y. y x : \sigma \rightarrow (\sigma \rightarrow \tau) \rightarrow \tau}^{intro \rightarrow}$$

В Haskell это `flip ($)`, или `(&)` из `Data.Function`: `:t \x y -> y x` даёт `a -> (a -> b) -> b`.

Функции в Haskell

Haskell — расширенное типизированное λ -исчисление

- Плюс примитивные типы, объявления имён, структуры данных и классы типов

```
ghci> (\x -> x + 1) 4
5
ghci> :t \x -> x + 1
```

```
\x -> x + 1 :: Int -> Int      -- на самом деле числа полиморфны, но об этом позже
```

- Функциям дают имена; сигнатуры на верхнем уровне пишут явно

```
f :: Int -> Int
f x = x + 1
```

- **Δ** Имя типа с маленькой буквы — **типовая переменная: параметрический полиморфизм**. `id` ничего не знает про аргумент, но результат обязан быть того же типа

```
id :: a -> a
id x = x
```

Функции высших порядков, операторы, рекурсия

-  Функции принимают функции — **функции высших порядков**
- Имя из специальных символов — **оператор**, применяется инфиксно

```
( $\$$ ) :: (a -> b) -> a -> b  
f  $\$$  x = f x
```

- Рекурсия с охранными выражениями для базового случая

```
factorial :: Int -> Int  
factorial n  
  | n < 1 = 1  
  | otherwise = n * factorial (n - 1)
```

 Что выведет `ghci> :t uncurry (flip const)`?

 Что выведет `ghci> :t first . first` при `first :: (a -> a') -> (a, b) -> (a', b)`?

 **Решение.**

```
uncurry (flip const) :: (a, c) -> c  
first . first :: (a -> a') -> ((a, b1), b2) -> ((a', b1), b2)
```

Аккумулирующий параметр

✎ Реализуйте факториал с помощью техники аккумулярующего параметра.

⚡ *Решение.*

```
factorial :: Int -> Int
factorial = go 1
  where
    go acc n
      | n < 1 = acc
      | otherwise = go (acc * n) (n - 1)
```

Данные в Haskell

Типы данных и паттерн-матчинг

```
data Animal
  = Cat String Int
  | Dog String
```

- **Δ** `Cat` и `Dog` — **конструкторы данных**: функции, реализованные языком; выделяют память и позиционно размещают компоненты

```
Cat :: String -> Int -> Animal
Dog :: String -> Animal
```

- Чтобы воспользоваться данными, значение **деконструируют** паттерн-матчингом: образец выбирается, если похож на то, как значение сконструировано, и его переменные начинают ссылаться на компоненты

```
describeAnimal :: Animal -> String
describeAnimal animal = case animal of
  Cat name nLives -> "This is cat " ++ name ++ " " ++ show nLives
  Dog name -> "This is dog " ++ name
```

- Записи с именованными полями порождают функции-аксессоры: `getName :: Penguin -> String`

```
data Penguin = Penguin { getName :: String, getAge :: Int }
```

Абстракция в типе

- Функции часто **частичны**: не для всех аргументов есть результат. Смоделируем это типом данных

```
data MaybeD = NothingD | JustD Double
safeSqrtD :: Double -> MaybeD
safeSqrtD x = if x < 0 then NothingD else JustD (sqrt x)
```

- — Так придётся объявлять свой **MaybeT** для каждого **T**
- ⇒ Haskell позволяет **абстрагироваться в типе**, как λ абстрагируется по значению в терме

```
data Maybe a = Nothing | Just a
safeSqrt :: Double -> Maybe Double
safeSqrt x = if x < 0 then Nothing else Just (sqrt x)
```

- **Δ** **Maybe** — не тип, а **типовый конструктор**: ему нужно передать типовой параметр, чтобы получить тип

Кайнды: типы для типов

- А что если дать типовому конструктору меньше параметров, чем он ждёт? Или больше?
-  **Система кайндов** контролирует корректность типовых аппликаций: простейшие «типы для типов»
- Обычные типы имеют кайнд $*$, типовые конструкторы — стрелочные кайнды; аппликация убирает одну стрелку

```
ghci> :k Int
Int :: *
ghci> :k Maybe
Maybe :: * -> *
ghci> :k Maybe Int
Maybe Int :: *
```

 Определите кайнд конструктора типа `data Free f a = Pure a | Free (f (Free f a))`.

 **Решение.**

`Free :: (* -> *) -> * -> *`: `f` применяется к типу, значит `f :: * -> *`; `a` стоит под `Pure`, значит `a :: *`.

Синонимы и обёртки

- **Синонимы типов** — просто другое имя для типа, если запись оригинала слишком длинная

```
type T a = VeryLongType Int (a -> AnotherLongType a)
```

- **newtype** — тип с одним конструктором и одним полем: не нужно выделять память под тег и ссылки на поля, значение обёртки — значение оборачиваемого типа, а сама обёртка существует только во время компиляции

```
newtype CourseId = CourseId Int64  
newtype ModuleId = ModuleId Int64
```

Классы типов

Классы типов: один идентификатор, разные реализации

- **Δ** Параметрический полиморфизм — один код для разных типов; классы типов — **один идентификатор для разных реализаций**, как перегрузка: механизм **специального (ad-hoc) полиморфизма**

```
class Eq a where
  (==) :: a -> a -> Bool
```

```
instance Eq CourseId where
  CourseId x == CourseId y = x == y

instance Eq a => Eq [a] where
  [] == [] = True
  x:xs == y:ys = x == y && xs == ys
  _ == _ = False
```

- Реализация выбирается по типу аргументов в месте вызова

```
ghci> CourseId 1 == CourseId 2
False
ghci> [CourseId 1, CourseId 2] == [CourseId 1, CourseId 2]
True
```

Классы типов как ограничение полиморфизма

- Параметрически-полиморфная функция ничего не может делать с аргументом, кроме как вернуть его или передать дальше
- Чтобы делать что-то ещё, нужна дополнительная информация о типе: `suc n = n + 1` не работает для строк, поэтому тип `suc :: a -> a` некорректен
- ➔ Классы типов — способ **ограничить** полиморфизм: не любой тип, а любой, для которого есть нужные операции

```
suc :: Num a => a -> a
suc n = n + 1
```

Задачи про классы типов

✎ Реализуйте функцию, проверяющую равенство всех элементов данного списка.

✎ Реализуйте инстанс полугруппы для функций.

✎ Реализуйте проверку равенства функций.

✚ Решение.

```
allEqual :: Eq a => [a] -> Bool
allEqual xs = and (zipWith (==) xs (drop 1 xs))

instance Semigroup b => Semigroup (a -> b) where    -- уже есть в base
    (f <> g) x = f x <> g x

instance (Enum a, Bounded a, Eq b) => Eq (a -> b) where    -- только для конечных областей
    f == g = all (\x -> f x == g x) [minBound .. maxBound]
```

Монады в Haskell

Функтор

- Δ `Functor` объявляется для **конструкторов типов** и заменяет все элементы контейнера, не меняя его структуру

```
class Functor (f :: * -> *) where
  fmap :: (a -> b) -> f a -> f b
```

```
instance Functor [] where
  fmap :: (a -> b) -> [a] -> [b]
  fmap _ [] = []
  fmap f (x:xs) = f x : fmap f xs
```

Эффекты как структура над результатом

- В Haskell функция просто вычисляет результат. Но часто нужно и «что-то ещё»: изменить состояние, написать в консоль — **побочные эффекты**
- Вернуть можно только результат, поэтому эффекты кодируем как **дополнительную структуру**, оборачивающую чистый результат: вместо `a` функция возвращает тип *вычислений* `f a`

Эффект	<code>f</code>
функция может завершиться ошибкой	<code>Maybe</code>
читает глобальное окружение <code>e</code>	<code>e -> _</code>
читает и обновляет состояние <code>s</code>	<code>s -> (s, _)</code>

- Стандартная библиотека даёт классы типов для значений вида `f a`: они позволяют работать с `a` внутри, как будто структуры `f` нет

Аппликативный функтор и монада

- **Applicative** позволяет писать выражения над вычислениями `f a`

```
class Functor f => Applicative (f :: * -> *) where
    pure :: a -> f a
    liftA2 :: (a -> b -> c) -> f a -> f b -> f c
instance Applicative Maybe where
    pure = Just
    liftA2 _ Nothing _ = Nothing
    liftA2 _ _ Nothing = Nothing
    liftA2 f (Just x) (Just y) = Just (f x y)
```

- **Monad** позволяет последовательно компоновать вычисления в императивном стиле

```
class Applicative m => Monad (m :: * -> *) where
    (>>=) :: m a -> (a -> m b) -> m b
newtype State s a = State { runState :: s -> (s, a) }
```

```
instance Monad (State s) where
    m >>= k = State \s ->
        let (s', x) = runState m s in
        runState (k x) s'
```

Состояние в императивном стиле

```
get :: State s s
get = State \s -> (s, s)

put :: s -> State s ()
put newS = State \_ -> (newS, ())

example :: State Int Int
example =
  get >>= \x ->
  put 42 >>= \() ->
  get >>= \y ->
  pure (x + y)
```

Трасса `runState example 1`:

- `get` при `s = 1` даёт `(1, 1): x = 1`
- `put 42` даёт `(42, ())`
- `get` при `s = 42` даёт `(42, 42): y = 42`
- `pure (x + y)` даёт `(42, 43)`

```
ghci> runState example 1
(42,43)
```

- Для таких цепочек есть **do-нотация**: тот же `example` строчка в строчку

```
example = do
  x <- get
  put 42
  y <- get
  pure (x + y)
```

Задачи про монады

✎ Реализуйте `liftA3` через `liftA2`.

✎ Реализуйте `>>=` через `join` и наоборот.

✎ Считайте два числа с консоли, поделите одно на другое нацело и распечатайте результат; если остаток ненулевой, распечатайте и его.

⚡ Решение.

```
liftA3 f a b c = liftA2 ($) (liftA2 f a b) c
```

```
m >>= k = join (fmap k m)
join mm = mm >>= id
```

```
main = do
  a <- readLn :: IO Int
  b <- readLn
  let (q, r) = a `divMod` b
  print q
  when (r /= 0) $ print r
```

Что мы вспомнили

термы	программа — дерево; вычисление — β -редукция, подстановка аргумента вместо параметра
типы	метки на вершинах по правилам; система типов — синтаксис меток и дроби; метка как множество
функции	λ , сигнатуры, типовые переменные, функции высших порядков, операторы
данные	конструкторы как функции, паттерн-матчинг, абстракция в типе, кайнды, <code>newtype</code>
классы	один идентификатор — разные реализации; ограничение полиморфизма
монады	эффект — структура над результатом; <code>Functor</code> , <code>Applicative</code> , <code>Monad</code> , <code>State</code> , <code>do</code>

- **Домашка fr2-01:** интерпретатор Lalalang — изучить выданный код, добавить операции, S-выражения, паттерн-матчинг. Минимум — задачи 1.1–1.2
- **На следующем занятии:** параметрический полиморфизм — глава 2

Материалы

Что почитать

- Конспект курса, глава 1 «Воспоминания о ФП» и Введение
- Philip Wadler, [Monads for functional programming](#) — классический текст про «эффект как структура»
- Graham Hutton, «Programming in Haskell» — если Haskell подзабылся
- Benjamin Pierce, «Types and Programming Languages», глава 9 — просто типизированное λ -исчисление и дроби
- [Typeclassopedia](#) — карта классов типов стандартной библиотеки